

WatchLockAI Comprehensive Research Report

This report provides a comprehensive overview of the research conducted for the development of the WatchLockAI cybersecurity platform. It covers all the research areas outlined in the initial research plan and provides detailed technical implementation guidance, recommended libraries and tools, code examples, and security best practices.

1. Windows Security APIs and Endpoint Protection Methods

This section details the various Windows APIs and techniques that can be leveraged for endpoint protection.

1.1. Event Tracing for Windows (ETW)

Description:

Event Tracing for Windows (ETW) is a high-speed, low-overhead, scalable tracing facility built into the Windows operating system. It provides a mechanism for applications and kernel-mode drivers to publish events, which can then be consumed by tracing tools and applications.

Key Features for WatchLockAI:

- **Real-time Monitoring:** ETW allows for real-time consumption of events, which is crucial for timely threat detection.
- **Kernel-Level Visibility:** ETW provides access to a wealth of kernel-level events, including process and thread creation, image loading, registry access, and network activity.
- **Low Overhead:** ETW is designed to have minimal performance impact on the system.

Implementation in C#:

The recommended approach for consuming ETW events in C# is to use the `.NET TraceProcessing` API. This API provides a powerful and flexible way to process ETW traces.

Recommended Library:

- `Microsoft.Windows.EventTracing.Processing.All`: This NuGet package contains the necessary libraries for working with the .NET TraceProcessing API.

Code Example (Conceptual):

```
using Microsoft.Windows.EventTracing;

// Create a trace session to consume real-time events
using (var session = new RealTimeTraceSession("WatchLockAI-Session"))
{
    // Enable the desired ETW providers
    session.EnableProvider(TraceLogProvider.Kernel);

    // Process events as they arrive
    session.Source.Process(traceEvent =>
    {
        // Analyze the event and look for suspicious activity
        if (traceEvent.EventName == "Process/Start")
        {
            // Handle process creation event
        }
    });
}
```

1.2. Windows Management Instrumentation (WMI)

Description:

WMI is the infrastructure for management data and operations on Windows-based

operating systems. It provides a unified way to access and manage system information and resources.

Key Features for WatchLockAI:

- **System Interrogation:** WMI can be used to query for a wide range of system information, including installed software, running processes, hardware details, and security settings.
- **Event Subscription:** WMI allows for subscribing to events, which can be used to monitor for changes in the system state.

Implementation in C#:

The `System.Management` namespace in .NET provides the classes for interacting with WMI.

Code Example (Conceptual):

```

using System.Management;

// Query for running processes
var query = new WqlObjectQuery("SELECT * FROM Win32_Process");
using (var searcher = new ManagementObjectSearcher(query))
{
    foreach (var process in searcher.Get())
    {
        // Analyze the process information
        Console.WriteLine(process["Name"]);
    }
}

// Subscribe to process creation events
var eventQuery = new WqlEventQuery("__InstanceCreationEvent",
    new TimeSpan(0, 0, 1),
    "TargetInstance ISA \"Win32_Process\"");

using (var watcher = new ManagementEventWatcher(eventQuery))
{
    watcher.EventArrived += (sender, e) =>
    {
        var newProcess =
            (ManagementBaseObject)e.NewEvent["TargetInstance"];
        // Handle the new process event
    };
    watcher.Start();
}

```

1.3. PowerShell Monitoring

Description:

PowerShell is a powerful scripting language and automation framework that is often

used by attackers. Monitoring PowerShell execution is crucial for detecting malicious activity.

Key Features for WatchLockAI:

- **Script Block Logging:** PowerShell can be configured to log all script blocks that are executed, providing a detailed record of PowerShell activity.
- **Module Logging:** PowerShell can also log events for the execution of PowerShell modules.
- **Transcription:** PowerShell can create a transcript of all PowerShell sessions, which can be useful for forensic analysis.

Implementation:

PowerShell logging can be enabled through Group Policy or by setting the appropriate registry keys.

1.4. Registry Monitoring

Description:

The Windows Registry is a hierarchical database that stores low-level settings for the operating system and for applications. Monitoring the registry for unauthorized changes is a key aspect of endpoint protection.

Key Features for WatchLockAI:

- **Real-time Change Detection:** It's important to be able to detect registry changes in real-time to prevent malicious modifications.

Implementation in C#:

The `Microsoft.Win32.Registry` class provides methods for accessing and modifying the registry. For real-time monitoring, you can use the `RegNotifyChangeKeyValue` Win32 API function. There are also libraries that wrap this functionality, such as the `RegistryMonitor` class found on CodeProject.

1.5. Process and Thread Monitoring

Description:

Monitoring the creation and activity of processes and threads is fundamental to endpoint security. Attackers often use malicious processes and threads to carry out their objectives.

Key Features for WatchLockAI:

- **Process Creation Events:** Detect the creation of new processes and analyze their command line arguments and parent processes.
- **Thread Creation Events:** Monitor for the creation of new threads within processes, which can indicate code injection.

Implementation in C#:

- **ETW:** ETW provides detailed events for process and thread creation, which is the recommended approach for real-time monitoring.
- `System.Diagnostics.Process` **Class:** This class can be used to enumerate running processes and get information about them.

1.6. Memory Scanning and Injection Detection

Description:

Attackers often use memory injection techniques to execute malicious code within the context of legitimate processes. Scanning process memory for signs of injection is a critical defense mechanism.

Key Techniques for WatchLockAI:

- **Analyze Memory Regions:** Look for memory regions with anomalous characteristics, such as `RWX` (Read, Write, Execute) permissions, which are often used for injected code.
- **Scan for PE Headers:** Scan process memory for Portable Executable (PE) headers, which can indicate the presence of an injected DLL.
- **Monitor API Calls:** Monitor for suspicious API calls that are often used for process injection, such as `CreateRemoteThread`, `WriteProcessMemory`, and `VirtualAllocEx`.

1.7. Windows Firewall Control

Description:

The Windows Firewall is a stateful host firewall that is built into the Windows operating system. Programmatically controlling the firewall is essential for enforcing network security policies.

Implementation in C#:

There are several ways to control the Windows Firewall programmatically:

- **netsh command-line tool:** This tool can be used to configure and manage the firewall.
- **Windows Firewall with Advanced Security API:** This COM-based API provides a rich set of interfaces for managing the firewall.
- **Third-party libraries:** There are several third-party libraries that provide a more convenient way to interact with the firewall API, such as `WindowsFirewallHelper`.

Recommended Library:

- `WindowsFirewallHelper`: This library provides a simple and intuitive way to manage Windows Firewall rules in C#.

Code Example (Conceptual):

```
using WindowsFirewallHelper;

// Block an application
var rule = FirewallManager.Instance.CreateApplicationRule(
    @"Block MyApp",
    FirewallAction.Block,
    @"C:\MyApp.exe");
FirewallManager.Instance.Rules.Add(rule);
```

2. MITRE ATT&CK Framework Integration Approaches

This section outlines how to integrate the MITRE ATT&CK framework into WatchLockAI for threat detection and analysis.

2.1. Real-time Mapping of System Events to ATT&CK Techniques

Description:

The core of MITRE ATT&CK integration is the ability to map system events to specific

ATT&CK techniques in real-time. This provides a structured way to understand and categorize adversary behavior.

Best Practices:

- **Start with Data Sources:** Begin by identifying the data sources that can provide evidence of ATT&CK techniques. These include Windows event logs, Sysmon logs, EDR telemetry, and network traffic.
- **Use a Structured Approach:** Follow a structured methodology for mapping events to techniques. The CISA document on ATT&CK mapping provides excellent guidance on this.
- **Leverage Open Source Tools:** Utilize open-source tools like Sigma rules and the MITRE Cyber Analytics Repository (CAR) to aid in the mapping process.
- **Validate Mappings:** It's crucial to validate the mappings to ensure their accuracy. This can be done through peer review and by comparing against known adversary behaviors.

2.2. Kill Chain Analysis

Description:

The Cyber Kill Chain is a model that describes the stages of a cyberattack. By mapping ATT&CK techniques to the different stages of the kill chain, you can gain a better understanding of the adversary's progress and intent.

Implementation:

- **Define Kill Chain Stages:** Define the stages of the kill chain that are relevant to your environment. The classic Lockheed Martin model is a good starting point.
- **Map Techniques to Stages:** Map the ATT&CK techniques to the corresponding stages of the kill chain.
- **Visualize the Kill Chain:** Create a visualization of the kill chain that shows the adversary's progress and the techniques that have been used.

2.3. Behavioral Detection Patterns

Description:

Instead of relying on signatures, behavioral detection focuses on identifying patterns of behavior that are indicative of malicious activity. This is a more effective way to detect novel and sophisticated threats.

Implementation:

- **Develop Behavioral Detectors:** Create detectors that look for specific patterns of behavior that are associated with ATT&CK techniques.
- **Use Machine Learning:** Leverage machine learning to identify anomalous behavior that may not be caught by traditional detection methods.
- **Correlate Events:** Correlate events from multiple sources to get a more complete picture of the adversary's activity.

2.4. Integration with Threat Intelligence Feeds and IOC Databases

Description:

Threat intelligence feeds and IOC (Indicator of Compromise) databases provide valuable information about known threats. Integrating this information into WatchLockAI can significantly improve its detection capabilities.

Implementation:

- **Consume Threat Intelligence Feeds:** Subscribe to threat intelligence feeds that provide information on the latest threats and adversary TTPs (Tactics, Techniques, and Procedures).
- **Use IOC Databases:** Integrate with IOC databases to check for the presence of known malicious indicators, such as file hashes, IP addresses, and domain names.
- **Automate IOC Matching:** Automate the process of matching system events against IOCs to enable real-time detection.

3. Tamperproofing and Anti-Evasion Techniques for Windows

This section covers techniques for protecting WatchLockAI from being tampered with or evaded by attackers.

3.1. Windows Service Hardening

Description:

Windows services are a common target for attackers. It's crucial to harden the WatchLockAI service to make it more resilient to attack.

Best Practices:

- **Run with Least Privilege:** The WatchLockAI service should run with the minimum set of privileges required to perform its functions.
- **Use a Service-Specific SID:** Assign a unique Service SID to the WatchLockAI service to isolate it from other services.
- **Implement Write-Restricted SIDs:** Use write-restricted SIDs to prevent the service from modifying non-service-related resources.
- **Restrict Network Access:** Define network restrictions for the service to limit its ability to communicate on the network.

3.2. Process Hiding and Obfuscation

Description:

Attackers often try to hide their processes to evade detection. WatchLockAI should be able to detect and prevent process hiding.

Techniques:

- **User-mode Hooking:** Attackers can hook API functions like `NtQuerySystemInformation` to filter out their processes from the process list.
- **Kernel-mode DKOM:** Direct Kernel Object Manipulation (DKOM) can be used to remove a process from the system's process list at the kernel level.

Detection:

- **Monitor for API Hooks:** Scan for hooks on critical API functions.
- **Kernel-level Integrity Checks:** Perform integrity checks on kernel data structures to detect modifications.

3.3. Anti-Debugging and Anti-Analysis

Description:

Attackers often use anti-debugging and anti-analysis techniques to make it more difficult for security researchers to analyze their malware.

Techniques:

- **Debugger Detection:** Check for the presence of a debugger using various techniques, such as the `IsDebuggerPresent` API function, checking for software breakpoints, and examining the process heap.
- **Timing Attacks:** Use timing attacks to detect the presence of a debugger.
- **Obfuscation:** Obfuscate the code to make it more difficult to understand.

Countermeasures:

- **Use a Stealthy Debugger:** Use a debugger that is designed to be difficult to detect.
- **Bypass Anti-Debugging Checks:** Patch out the anti-debugging checks in the malware.
- **Deobfuscate the Code:** Use deobfuscation techniques to make the code more readable.

4. LLM Integration for Real-Time Threat Analysis

This section discusses how to integrate a Large Language Model (LLM) into WatchLockAI for real-time threat analysis.

4.1. Local LLM Deployment

Description:

For performance and privacy reasons, it's desirable to deploy the LLM locally on the endpoint.

Strategies and Tools:

- **ONNX Runtime:** ONNX Runtime is a high-performance inference engine that can be used to run LLMs on a variety of hardware.
- **Quantization:** Quantize the LLM to reduce its size and memory footprint.
- **Pruning:** Prune the LLM to remove unnecessary weights and further reduce its size.

4.2. Memory Management

Description:

LLMs can be very memory-intensive. It's important to use efficient memory management techniques to minimize the impact on the endpoint.

Techniques:

- **PagedAttention:** PagedAttention is a memory management technique that is inspired by virtual memory and paging in operating systems. It can significantly reduce the memory footprint of LLMs.
- **MemGPT:** MemGPT is a virtual memory management system for LLMs that allows them to manage their own memory.

4.3. Real-time Log Analysis with NLP

Description:

NLP can be used to analyze security logs in real-time to identify potential threats.

Techniques:

- **Log Parsing:** Parse security logs to extract relevant information, such as timestamps, source IP addresses, and event IDs.
- **Anomaly Detection:** Use NLP to identify anomalous log entries that may indicate a security threat.
- **Threat Classification:** Classify threats based on the information in the logs.

5. Enterprise Security Tool APIs and Integration

This section covers how to integrate WatchLockAI with other enterprise security tools.

5.1. Microsoft Defender for Endpoint

API:

Microsoft Defender for Endpoint provides a rich set of APIs for interacting with the service. These APIs can be used to retrieve information about alerts, devices, and users, as well as to take response actions, such as isolating a device or quarantining a file.

Integration:

- **Alert Ingestion:** Ingest alerts from Defender for Endpoint into WatchLockAI for correlation and analysis.
- **Response Actions:** Use the Defender for Endpoint APIs to take response actions from within the WatchLockAI console.

5.2. Azure Sentinel

API:

Azure Sentinel provides a REST API for interacting with the service. This API can be used to create and manage data connectors, analytic rules, incidents, and bookmarks.

Integration:

- **Data Ingestion:** Send security data from WatchLockAI to Azure Sentinel for analysis and correlation.
- **Incident Management:** Create and manage incidents in Azure Sentinel based on alerts from WatchLockAI.

5.3. Third-Party EDR and SIEM Integration

APIs:

Most EDR and SIEM solutions provide APIs for integration. These APIs can be used to send and receive data, as well as to take response actions.

Integration Patterns:

- **Data Forwarding:** Forward security data from WatchLockAI to the SIEM in a standard format, such as CEF or LEEF.
- **API-based Integration:** Use the APIs of the EDR and SIEM solutions to create a more tightly integrated solution.

Recommended Libraries:

- **CrowdStrike:** `falconpy` (Python SDK)
- **SentinelOne:** There are several community-supported Python libraries available on GitHub.

6. Windows 11 Service Development and Deployment

This section provides guidance on how to develop and deploy the WatchLockAI agent as a Windows service.

6.1. C#/.NET Windows Service Best Practices

Best Practices:

- **Use the `BackgroundService` Class:** The `BackgroundService` class in .NET Core and .NET 5+ provides a convenient way to create long-running services.
- **Handle Exceptions Gracefully:** Make sure to handle all exceptions to prevent the service from crashing.
- **Use a Logging Framework:** Use a logging framework, such as Serilog or NLog, to log important events and errors.
- **Implement a Health Check:** Implement a health check endpoint that can be used to monitor the health of the service.

6.2. MSI Installer Creation

Tools:

- **Visual Studio Installer Projects:** This extension for Visual Studio can be used to create MSI installers.
- **WiX Toolset:** The WiX Toolset is a powerful open-source tool for creating MSI installers.
- **Advanced Installer:** Advanced Installer is a commercial tool that provides a user-friendly interface for creating MSI installers.

Best Practices:

- **Install the Service:** The installer should install the WatchLockAI service and configure it to start automatically.
- **Request Administrator Privileges:** The installer should request administrator privileges to be able to install the service.
- **Provide an Uninstaller:** The installer should also provide an uninstaller that removes the service and all of its files.